Simon Edmunds

12590227

CS 5480: Deep Learning
Assignment 2
Total: 100 (+20 bonus)
Due: March 12, 2026 at 11:59 PM

Instructions

- This homework has two parts: **Part I (Theory)** and **Part II (Coding)**
- Submit a PDF with theory answers and a separate Python notebook (.ipynb) for coding questions
- Show all work for theory questions
- Include comments and markdown explanations in your code
- For coding questions, use NumPy for numerical computations (no PyTorch/TensorFlow)

Submission & Formatting Requirements

- **Submission platform:** All submissions **must be uploaded to Canvas** by the stated deadline. Email submissions will **not** be accepted under normal circumstances.
- **Single ZIP file:** You must submit **one ZIP file** containing:
    - A **typed PDF** with all **theory answers**
    - A completed **Jupyter notebook (.ipynb)** for the coding portion

PART I: THEORY

1. From MLE to MSE (25 points)

Every time you train a neural network with mean squared error, you are implicitly making a probabilistic bet: that the world is Gaussian. This is rarely stated explicitly, and even more rarely justified. But it turns out that MSE is not just a convenient formula someone invented—it is the *unique* loss function that falls out of maximum likelihood estimation when you assume your targets are corrupted by independent Gaussian noise. Change that assumption, and you get a different loss.

This question walks you through that derivation from first principles. By the end, you will have proven that minimizing MSE and maximizing a Gaussian likelihood are exactly the same optimization problem in disguise—and you will understand precisely which assumptions make that equivalence valid, and what breaks when they are violated. Along the way, you will also derive a closed-form estimate for the noise level in your data, which turns out to be nothing more than the average squared residual at convergence.

Setup

Suppose we have a neural network $y(x, w)$ that maps an input $x \in R^D$ to a vector of K predictions $y \in R^K$. Let $X = \{x_n\}_{n=1}^N$ and $T = \{t_n\}_{n=1}^N$ denote the full training inputs and targets, where each $t_n \in R^K$.

We model each target vector as a noisy observation of the network output:

$$p(t|x, w) = N\left(t \mid y(x, w), \sigma^2 I\right)$$

where σ^2 is a shared noise variance and I is the $K \times K$ identity matrix. This says each target component is independently corrupted by Gaussian noise with the same variance σ^2. The network's job is to predict the clean signal; the noise is everything it cannot explain.

Note: You don't need prior experience with multivariate Gaussians — all the matrix algebra you need is provided in the hints.

Part (a): Writing Out the Likelihood (5 points)

Before we can maximize anything, we need to write down what we are maximizing. The likelihood is the probability of observing our training targets given the model parameters—it measures how well the current weights explain the data.

(i) Write out the full expression for the multivariate Gaussian $p(t|x,w) = N(t|\mu,\Sigma)$ by substituting $\mu = y(x,w)$ and $\Sigma = \sigma^2 I$. Your answer should be an explicit formula involving an exponential.

Hint: Recall that for a K-dimensional Gaussian,

$$N(t|\mu,\Sigma) = \frac{1}{(2\pi)^{K/2}|\Sigma|^{1/2}} exp\left(-\frac{1}{2}(t-\mu)^{\top}\Sigma^{-1}(t-\mu)\right).$$

My solution:

$$p(t|x,w) = \frac{1}{(2\pi)^{K/2}|\sigma^2 I|^{1/2}} exp\left(-\frac{1}{2}(t-y(x,w))^{\top}(\sigma^2 I)^{-1}(t-y(x,w))\right)$$

(ii) The expression from part (i) can be simplified considerably by exploiting the special structure of $\Sigma = \sigma^2 I$. Compute $|\sigma^2 I|$ and $(\sigma^2 I)^{-1}$, then simplify the quadratic form in the exponent.

Hint: The determinant of $\sigma^2 I$ is $(\sigma^2)^K$. The inverse is $(\sigma^2 I)^{-1} = \frac{1}{\sigma^2} I$. Also recall that for any vector v, $v^{\top} v = \| v\|^2$, so:

$$(t-y(x,w))^{\top}(\sigma^2 I)^{-1}(t-y(x,w)) = \frac{1}{\sigma^2}\| t-y(x,w)\|^2.$$

$$p(t|x,w) = \frac{1}{(2\pi)^{K/2}(\sigma^2)^{K/2}} exp\left(-\frac{1}{2}(t-y(x,w))^{\top}(\frac{1}{\sigma^2}I)(t-y(x,w))\right)$$

$$p(t|x,w) = \frac{1}{(2\pi)^{K/2}(\sigma^2)^{K/2}} exp\left(-\frac{1}{2\sigma^2}(t-y(x,w))^{\top}(t-y(x,w))\right)$$

$$p(t|x,w) = \frac{1}{(2\pi\sigma^2)^{K/2}} exp\left(-\frac{1}{2\sigma^2}||t-y(x,w)||^2\right)$$

Part (b): The Log-Likelihood (7 points)

Products are hard to work with. Sums are not. Taking the log turns the product over training examples into a sum, and since log is monotonically increasing, maximizing the log-likelihood is identical to maximizing the likelihood itself. This is one of the most useful tricks in all of probabilistic machine learning.

Assuming the N training examples are independent, the full likelihood is:

$$p(T|X,w) = \prod_{n=1}^{N} p(t_n|x_n,w)$$

(i) Take the log of this expression to obtain the log-likelihood $lnp(T|X,w)$. Write your answer as a sum over training examples.

$$ln\, p(T|X,w) = ln \prod_{n=1}^{N} p(t_n|x_n,w)$$

$$ln\, p(T|X,w) = \sum_{n=1}^{N} ln\, p(t_n|x_n,w)$$

(ii) Substitute your simplified Gaussian expression from Part (a). You should obtain something of the form:

$$lnp(T|X,w) = \underbrace{-\frac{NK}{2}ln(2\pi) - \frac{NK}{2}ln\sigma^2}_{terms\ independent\ of\ w} - \frac{1}{2\sigma^2}\sum_{n=1}^{N} \| y(x_n,w) - t_n\|^2$$

Show the algebraic steps to arrive at this form. You may treat $-\frac{NK}{2}ln(2\pi)$ as a constant.

$$p(t_n|x_n,w) = \frac{1}{(2\pi\sigma^2)^{K/2}} exp\left(-\frac{1}{2\sigma^2}||t_n - y(x_n,w)||^2\right)$$

$$ln\, p(t_n|x_n,w) = ln(\frac{1}{(2\pi\sigma^2)^{K/2}}) + ln(exp\left(-\frac{1}{2\sigma^2}||t_n - y(x_n,w)||^2\right))$$

$$ln\, p(t_n|x_n,w) = -\frac{K}{2}ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}||t_n - y(x_n,w)||^2$$

$$ln\, p(t_n|x_n,w) = -\frac{K}{2}ln(2\pi) - \frac{K}{2}ln(\sigma^2) - \frac{1}{2\sigma^2}||t_n - y(x_n,w)||^2$$

$$ln\, p(T|X,w) = \sum_{n=1}^{N}\left[-\frac{K}{2}ln(2\pi) - \frac{K}{2}ln(\sigma^2) - \frac{1}{2\sigma^2}||t_n - y(x_n,w)||^2\right]$$

$$ln\, p(T|X,w) = -\frac{NK}{2}ln(2\pi) - \frac{NK}{2}ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}||t_n - y(x_n,w)||^2$$

(iii) Look at the expression you derived. Which terms depend on w? Which terms are constants that play no role in the optimization? Being able to identify and discard irrelevant terms is a core skill in deriving learning algorithms.

Only the squared error term $\sum_{n=1}^{N}||t_n - y(x_n,w)||$ depends on w because $y(x_n,w)$ depends on the network weights. Both of the other two terms $-\frac{NK}{2}ln(2\pi) - \frac{NK}{2}ln(\sigma^2)$ are constants, and can be discarded without affecting the optimization process.

Part (c): From Likelihood Maximization to Loss Minimization (6 points)

We now arrive at the central result. The log-likelihood you derived in Part (b) looks complicated, but most of it is clutter. Stripping away everything that does not depend on w reveals that maximizing the likelihood is identical to minimizing a much simpler objective—one you have been using all along without its probabilistic justification.

(i) Explain why *maximizing* the log-likelihood with respect to w is equivalent to *minimizing* the sum-of-squares error:

$$E(w) = \frac{1}{2}\sum_{n=1}^{N} \| y(x_n, w) - t_n \|^2$$

From part b above, I derived the formula for the log-likelihood:

$$ln\, p(T|X, w) = -\frac{NK}{2} ln(2\pi) - \frac{NK}{2} ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N} ||t_n - y(x_n, w)||^2$$

As I mentioned in part iii of question b, the first two terms $-\frac{NK}{2} ln(2\pi) - \frac{NK}{2} ln(\sigma^2)$ are constants, and because they do not depend on w, they will remain constants during optimization. Because they are constants, adding them will not affect the argmax of the function. Similarly, the factor of σ^{-2} can be ignored, as multiplying by a positive constant will also not affect the argmax. This leaves us with the expression:

$$ln\, p(T|X, w) = -\frac{1}{2}\sum_{n=1}^{N} ||t_n - y(x_n, w)||^2$$

This expression shows us that maximizing the log-likelihood with respect to w is the same as minimizing the value of the expression $\frac{1}{2}\sum_{n=1}^{N} ||t_n - y(x_n, w)||^2$ which matches the definition of the sum-of-squares error.

Be precise: which terms can be dropped and why? You may use the fact that adding a constant or multiplying by a positive constant does not change the $argmax$ or $argmin$.

(ii) Notice that the log-likelihood from Part (b) contains the factor $-\frac{1}{2\sigma^2}$ multiplying the sum of squared residuals. Does the value of σ^2 change the maximizing w^*? This is a subtle but important point: the noise level in your data affects how confident you are in your predictions, but does it affect which weights are optimal?

Hint: Think about what happens to the $argmax$ when you scale an objective function by a positive constant.

As explained above, multiplying by a positive constant σ^{-2} will not affect the $argmax$ of this function, and thus it will not affect which weights are optimal.

(iii) The derivation above justifies MSE, but only under specific assumptions. State **one** modeling assumption from the list below that makes this justification valid, and give one concrete failure mode if that assumption is violated. Thinking carefully about when MSE is *inappropriate* is just as important as knowing how to use it.

- Gaussian noise on the targets
- Independence across training examples
- Independence across output dimensions (given x and w)
- Shared noise variance across output dimensions (homoscedasticity)

This derivation assumes that the targets are affected by Gaussian noise. This can cause a failure if the noise contains large outliers or spikes, because squared error will penalize these large error spikes very strongly, causing the model to overfit to those outliers.

Part (d): Estimating the Noise Variance (7 points)

So far we have treated σ^2 as a fixed but unknown constant and optimized over w alone. But once we have found the optimal weights w^*, we can ask a natural follow-up question: given that the network is as good as it can be, how noisy is the data it is trying to fit? It turns out that σ^2 also has a clean maximum likelihood estimate—and the formula is both elegant and intuitive.

(i) Write the log-likelihood from Part (b) explicitly as a function of σ^2, treating $w = w^*$ as fixed. Let $r_n = y(x_n, w^*) - t_n$ denote the residual for example n—the part of the target that the network failed to predict. You should arrive at:

$$lnp(T|X, w^*, \sigma^2) = -\frac{NK}{2} ln\sigma^2 - \frac{1}{2\sigma^2} \sum_{n=1}^{N} \| r_n \|^2 + const$$

$$ln\, p(T|X, w^*) = -\frac{NK}{2} ln(2\pi) - \frac{NK}{2} ln(\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^{N} ||t_n - y(x_n, w^*)||^2$$

$$ln\, p(T|X, w^*, \sigma^2) = -\frac{NK}{2} ln(2\pi) - \frac{NK}{2} ln(\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^{N} ||r_n||^2$$

$-\frac{NK}{2} ln(2\pi)$ does not depend on σ^2, so it's a constant

$$ln\, p(T|X, w^*, \sigma^2) = -\frac{NK}{2} ln(\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^{N} ||r_n||^2 + const$$

where $const$ absorbs any terms that do not depend on σ^2.

Hint: Keep both the $ln\sigma^2$ term and the $\frac{1}{\sigma^2}$ term—dropping either one will give the wrong answer in the next part.

(ii) To find the MLE for σ^2, differentiate the log-likelihood with respect to σ^2 and set the derivative to zero. Show your differentiation steps carefully. This is a single-variable calculus problem: treat σ^2 as one variable throughout.

Hint: Recall that $\frac{d}{d\sigma^2} ln\sigma^2 = \frac{1}{\sigma^2}$ *and* $\frac{d}{d\sigma^2}\frac{1}{\sigma^2} =- \frac{1}{(\sigma^2)^2}$.

$$L(\sigma^2) = ln\, p(T|X, w^*, \sigma^2) =- \frac{NK}{2} ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N} ||y(x_n, w^*) - t_n||^2 + const$$

$$\text{Let } S = \sum_{n=1}^{N} ||y(x_n, w^*) - t_n||^2$$

$$\frac{dL}{d\sigma^2} = \frac{d}{d\sigma^2}\left(- \frac{NK}{2} ln(\sigma^2)\right) + \frac{d}{d\sigma^2}\left(- \frac{S}{2\sigma^2}\right)$$

$$\frac{dL}{d\sigma^2} =- \frac{NK}{2\sigma^2} + \frac{d}{d\sigma^2}\left(- \frac{S}{2\sigma^2}\right)$$

$$\frac{dL}{d\sigma^2} =- \frac{NK}{2\sigma^2} + \frac{S}{2(\sigma^2)^2}$$

$$- \frac{NK}{2\sigma^2} + \frac{S}{2(\sigma^2)^2} = 0$$

(iii) Show that solving the equation from part (ii) yields:

$$\sigma_*^2 = \frac{1}{NK}\sum_{n=1}^{N} \| \, y(x_n, w^*) - t_n \|^2$$

$$-\frac{NK}{2\sigma^2} + \frac{S}{2(\sigma^2)^2} = 0$$

$$-NK\sigma^2 + S = 0$$

$$S = NK\sigma^2$$

$$\sigma^2 = \frac{S}{NK}$$

Putting S back in:

$$\sigma^2 = \frac{1}{NK}\sum_{n=1}^{N} ||y(x_n, w^*) - t_n||^2$$

(iv) Step back and look at what this formula says. You started from a probabilistic model, derived a training objective, optimized it, and then asked what the optimal noise estimate is. The answer turned out to be the average squared residual—the mean of the errors your network makes at convergence. Interpret this result in plain language: what is σ_*^2 actually measuring, and why is it exactly what you would expect a well-calibrated estimate of data noise to look like?

σ^2 measures the average squared error on the predictions after the network has been fully trained. Basically, if the model already explains all of the predictable structure in the data, then whatever error is left over must come from noise in the data. Calculating σ^2 gives an estimate of the noise in the data.

2. Why Mini-Batch Gradient Descent Works (25 points)

When training a neural network, one of the most fundamental questions is deceptively simple: *how much data should we look at before taking a gradient step?* At one extreme, we could compute the exact gradient over the entire dataset—but for modern datasets with millions of examples, this is computationally prohibitive. At the other extreme, we could update the weights after every single data point—but a gradient computed from one example is an extremely noisy estimate of the true gradient, and training becomes erratic. Mini-batch gradient descent is the practical middle ground: at each iteration, we draw a small random subset of the data, called a **mini-batch**, compute the gradient on that subset, and take a step.

But how do we choose the right batch size? And how much does a larger batch actually help? It turns out the answer lies in a classical result from statistics: the error of a sample mean decreases as $\sigma/\sqrt{N}$, where N is the sample size. This question guides you through deriving that result from scratch, building the intuition for why it matters, and connecting it back to the practical decisions we make when training neural networks.

Setup

Let $x_1, \ldots, x_N$ be a set of values drawn **independently** from a distribution with mean μ and variance σ^2. You can think of each x_n as the gradient contribution from a single data point—a noisy measurement of the true underlying gradient. Define the **sample mean**:

$$\bar{x} = \frac{1}{N} \sum_{n=1}^{N} x_n.$$

Our goal is to understand how well $\bar{x}$ estimates the true mean μ as N grows, and what this implies for gradient estimation in mini-batch training.

Part (a): Expectation of the Sample Mean (5 points)

Before measuring how far $\bar{x}$ is from μ on average, we first check whether $\bar{x}$ is even pointing in the right direction.

(i) Show that the expectation of the sample mean equals the true mean:

$$E[\bar{x}] = \mu.$$

Hint: Use linearity of expectation. Recall that $E[x_n] = \mu$ *for all* n.

$$\bar{x} = \frac{1}{N}\sum_{n=1}^{N} x_n$$

$$E[\bar{x}] = E\left[\frac{1}{N}\sum_{n=1}^{N} x_n\right]$$

$$E[\bar{x}] = \frac{1}{N}E\left[\sum_{n=1}^{N} x_n\right]$$

$$E[\bar{x}] = \frac{1}{N}\sum_{n=1}^{N} E\left[x_n\right]$$

$$E[\bar{x}] = \frac{1}{N}\sum_{n=1}^{N} \mu$$

$$E[\bar{x}] = \frac{1}{N}(N\mu)$$

$$E[\bar{x}] = \mu$$

(ii) An estimator satisfying $E[\bar{x}] = \mu$ is called **unbiased**. Why is this a desirable property for a gradient estimator in mini-batch training? What would go wrong during training if the gradient estimator were systematically biased in some direction?

An unbiased estimator means that the expected value of the gradient always equals the true gradient. This means that on average, each mini-batch gradient will point in the correct direction for optimizing. If there were bias, then it would make the gradients push in the wrong direction.

Part (b): Variance of the Sample Mean (8 points)

Knowing that x^{-} is unbiased tells us we are pointing in the right direction *on average*, but it says nothing about how noisy individual estimates are. We now quantify this by computing the expected squared error $E[(x^{-} - \mu)^2]$.

(i) Expand $(x^{-} - \mu)^2$ by substituting the definition of x^{-}. Show that:

$$(x^{-} - \mu)^2 = \frac{1}{N^2}\left(\sum_{n=1}^{N}(x_n - \mu)\right)^2.$$

$$x^{-} = \frac{1}{N}\sum_{n=1}^{N} x_n$$

$$x^{-} - \mu = \frac{1}{N}\sum_{n=1}^{N} x_n - \mu$$

Rewriting μ as a sum:

$$x^{-} - \mu = \frac{1}{N}\sum_{n=1}^{N} x_n - \frac{1}{N}\sum_{n=1}^{N} \mu$$

$$x^{-} - \mu = \frac{1}{N}\sum_{n=1}^{N}\left(x_n - \mu\right)$$

$$(x^{-} - \mu)^2 = \left(\frac{1}{N}\sum_{n=1}^{N}\left(x_n - \mu\right)\right)^2$$

$$(x^{-} - \mu)^2 = \frac{1}{N^2}\left(\sum_{n=1}^{N}\left(x_n - \mu\right)\right)^2$$

Hint: Write $x^{-} - \mu = \frac{1}{N}\sum_{n=1}^{N}(x_n - \mu)$ *and square both sides.*

(ii) Expand the square of the sum. You will obtain two types of terms: squared terms $(x_n - \mu)^2$ and cross terms $(x_n - \mu)(x_m - \mu)$ for $n \neq m$. Write out the expansion explicitly, separating these two types.

$$(x^{-} - \mu)^2 = \frac{1}{N^2}\left(\sum_{n=1}^{N}(x_n - \mu)\right)^2$$

$$(x^{-} - \mu)^2 = \frac{1}{N^2}\left[\sum_{n=1}^{N}(x_n - \mu)^2 + \sum_{n\neq m}(x_n - \mu)(x_m - \mu)\right]$$

(iii) Take the expectation of your expansion. Use the following two facts:

- $E[(x_n - \mu)^2] = \sigma^2$ for all n.
- $E[(x_n - \mu)(x_m - \mu)] = 0$ for $n \neq m$, since the samples are drawn independently (so their centered products have zero expectation).

Show step by step that:

$$E\left[(x^{-} - \mu)^2\right] = \frac{\sigma^2}{N}.$$

$$E\left[(x^{-} - \mu)^2\right] = \frac{1}{N^2}\left[E\left(\sum_{n=1}^{N}(x_n - \mu)^2\right) + E\left(\sum_{n\neq m}(x_n - \mu)(x_m - \mu)\right)\right]$$

$$E\left[(x^{-} - \mu)^2\right] = \frac{1}{N^2}\left[\sum_{n=1}^{N}E\left((x_n - \mu)^2\right) + \sum_{n\neq m}E\left((x_n - \mu)(x_m - \mu)\right)\right]$$

$$E\left[(x^{-} - \mu)^2\right] = \frac{1}{N^2}\left[\sum_{n=1}^{N}\sigma^2 + \sum_{n\neq m}0\right]$$

$$E\left[(x^{-} - \mu)^2\right] = \frac{1}{N^2}\sum_{n=1}^{N}\sigma^2$$

$$E\left[(x^{-} - \mu)^2\right] = \frac{1}{N^2}\left(N\sigma^2\right)$$

$$E\left[(x^{-} - \mu)^2\right] = \frac{\sigma^2}{N}$$

(iv) The **root mean square (RMS) error** of the sample mean is defined as $\sqrt{E[(x^{-} - \mu)^2]}$. Using your result from part (iii), write down the RMS error and confirm it equals $\sigma/\sqrt{N}$. Why does this quantity—rather than the raw variance σ^2/N—give a more intuitive sense of the typical estimation error?

$$E\left[(x^{-} - \mu)^2\right] = \frac{\sigma^2}{N}$$

$$\sqrt{E\left[(x^{-} - \mu)^2\right]} = \sqrt{\frac{\sigma^2}{N}}$$

$$\sqrt{E\left[(x^{-} - \mu)^2\right]} = \frac{\sigma}{\sqrt{N}}$$

The raw variance result is in terms of squared units (σ^2), which are harder to interpret. RMS converts that into the same units as the original variable, which makes it much easier to interpret.

Part (c): Diminishing Returns of Larger Batches (6 points)

The result $\sigma/\sqrt{N}$ has a striking practical implication: doubling the batch size does *not* halve the gradient noise. The improvement is sublinear—you have to work much harder for each incremental gain in accuracy.

(i) Suppose we increase the batch size by a factor of 100, from N to $100N$. By what factor does the RMS error in the gradient estimate decrease? Show your calculation explicitly.

$$\frac{\sigma}{\sqrt{100N}}$$

$$\frac{\sigma}{\sqrt{100}\sqrt{N}}$$

$$\frac{\sigma}{10\sqrt{N}}$$

$$\frac{1}{10}\left(\frac{\sigma}{\sqrt{N}}\right)$$

The RMS error decreases by a factor of $\frac{1}{10}$

(ii) Fill in the table below to illustrate the diminishing returns of increasing batch size. For each value of N, compute the RMS error relative to the $N = 1$ baseline (i.e. express the error as a multiple of σ, rounded to three decimal places). The first two rows are completed as examples.

Batch size N	RMS error
1	$\sigma = 1.000\sigma$
4	$\sigma/2 = 0.500\sigma$
16	$\sigma/4 = 0.250\sigma$
64	$\sigma/8 = 0.125\sigma$
256	$\sigma/16 = 0.063\sigma$
1024	$\sigma/32 = 0.031\sigma$

(iii) Based on the table and your result in part (i), explain in 2–3 sentences why practitioners rarely use very large batch sizes, even though a larger batch gives a more accurate gradient estimate. What is the practical tradeoff between gradient accuracy and computational cost?

Increasing the batch size reduces the gradient noise, but the amount of improvement decreases quickly with increasing batch size. Because computing gradients with larger batches requires significantly more memory, practitioners are more likely to use a moderate batch size to balance accuracy and processing time/resources.

Part (d): Connecting to Mini-Batch Gradient Descent (6 points)

We now connect the statistical result to the mechanics of mini-batch training and the practical decisions that go into running it.

(i) In mini-batch gradient descent, we estimate the true gradient $\nabla E(w)$ using a mini-batch of size B:

$$\nabla E(w) \approx \frac{1}{B} \sum_{n \in B} \nabla E_n(w)$$

where B is the set of indices in the mini-batch and $\nabla E_n(w)$ is the gradient contribution from a single data point. Using the result from Part (b), write down the RMS error of this gradient estimate as a function of B. Assume the per-sample gradient contributions $\nabla E_n(w)$ are drawn i.i.d. with standard deviation σ_g, where σ_g denotes the standard deviation of the per-sample gradient along a single coordinate.

$$RMS = \frac{\sigma_g}{\sqrt{B}}$$

(ii) The statistical result in Part (b) relies critically on the assumption that the x_n are drawn **independently**—so their centered products have zero expectation, as stated in Part (b)(iii). In the context of mini-batch training, this means mini-batches must be sampled randomly from the dataset. Explain why this matters: what kind of correlations might exist in a raw, unshuffled dataset, and how would those correlations affect the quality of the gradient estimate? Describe one standard practice used to address this.

The statistical result requires that the samples are independent, which means that mini batches must be sampled randomly. This matters, because if a mini-batch contains many correlated (not fully independent) samples, the gradient will become biased toward that subset of the data, instead of fitting the true gradient well. In real-world data, there are often correlations baked into the dataset (consecutive frames in a video, or time series data). A standard practice is to shuffle the dataset to address and mitigate this correlation.

(iii) In practice, mini-batch sizes are often chosen to be powers of 2 (e.g. 64, 128, 256). This choice is driven by hardware considerations, not by the statistical analysis above. Give one reason why powers of 2 are preferred on modern hardware accelerators such as GPUs. Then, given the diminishing returns from Part (c), which of the following batch sizes would you choose if a single gradient evaluation takes time proportional to B, and briefly justify your answer:

$$B \in \{1,\ 32,\ 256,\ 4096\}.$$

Powers of two match well with modern hardware/memory layouts, as well as with parallelizing across threads. This allows hardware to process data more efficiently and balance the load cleanly.

Of the provided options, I would choose B = 32, because it significantly reduces noise compared to B = 1, while still being much easier and faster to compute than the larger batches.

PART II: CODING

3. Backpropagation from Scratch (50 points)

Automatic differentiation libraries like PyTorch and TensorFlow compute gradients for you with a single call to `.backward()`. But what is actually happening inside that call? In this problem you will find out by implementing the full forward and backward passes of a neural network using **NumPy only** — no autograd, no PyTorch, no TensorFlow.

By the end you will have a working MLP trained entirely by hand-coded gradient descent, and you will have verified that your gradients are numerically correct. This is the single most effective exercise for building a genuine understanding of how backpropagation works.

The notebook uses two datasets intentionally. Parts (a)–(e) use the synthetic `make_moons` dataset, whose simple 2D geometry makes it easy to verify that your implementation is correct before moving to real data. Parts (f)–(h) use the **Wisconsin Breast Cancer** dataset: 569 tumour measurements, 30 features, binary label (malignant / benign). This is where your network does something that matters — and where the asymmetric costs you analysed in the theory section become concrete.

All training in this assignment uses **full-batch gradient descent** (no mini-batching). The statistical justification for mini-batching is covered in Theory Question 3; here we keep the training loop simple so the focus stays on getting the gradients right.

Key questions you will answer:

- How do gradients flow backward through a sigmoid, a ReLU, and a linear layer?
- How do you verify that an analytic gradient implementation is correct?
- How does learning rate affect convergence, and what does divergence look like?
- How does model capacity (hidden layer size) affect the learned decision boundary on real data?
- What is the practical difference between a wide shallow network and a narrow deep network of the same parameter count?

Tasks (see notebook for details)

Part (a): Forward Pass (6 points)
Implement `relu`, `sigmoid`, and `forward` for the `MLP` class. The forward pass must cache intermediate values (`z1`, `h`, `z2`, `y_hat`) needed by the backward pass. A provided sanity check verifies output shape and range.

Part (b): Loss and Output Gradient (4 points)
Implement `binary_cross_entropy` and `bce_grad`. A provided sanity check verifies that

near-perfect predictions give near-zero loss and confident wrong predictions give high loss.

Part (c): Backward Pass (14 points)
Implement `backward` using the upstream × local gradient pattern from Lecture 10A. The computation proceeds in three scaffolded steps:

- **(c.1)** Output layer: sigmoid derivative → `dz2`, `dW2`, `db2`, `dh` (5 pts)
- **(c.2)** ReLU: indicator derivative → `dz1` (4 pts)
- **(c.3)** Hidden layer: `dW1`, `db1` (5 pts)

Implementation Hint: When computing bias gradients (`db1`, `db2`), pay close attention to NumPy's `axis` and `keepdims` arguments. A common bug is summing over the batch dimension without `keepdims=True`, which results in a rank-1 array that broadcasts incorrectly during the parameter update. A provided shape-check cell verifies that every gradient has the correct dimensions before you proceed.

Part (d): Gradient Check (6 points)
Verify your analytic gradients against numerical finite differences:

$$\frac{\partial E}{\partial w_i} \approx \frac{E(w_i+\varepsilon)-E(w_i-\varepsilon)}{2\varepsilon}, \qquad \varepsilon = 10^{-5}.$$

A provided function computes relative errors and prints a pass/fail table. All relative errors must be below 10^{-4} before continuing. Small discrepancies near this threshold are expected due to floating-point precision and do not indicate a bug. In your writeup, report the errors for each parameter and describe any fixes you made to `backward`.

Part (e): Training Loop (6 points)
Implement full-batch gradient descent on `make_moons`. Plot the training loss curve over 1000 epochs and confirm that it decreases smoothly. In your writeup, explain what a smoothly decreasing curve tells you about the correctness of your implementation.

Part (f): Decision Boundary Visualization — Breast Cancer (6 points)
Train two networks on the full 30-dimensional breast cancer features (`hidden_dim` $\in \{4, 64\}$, `lr`$= 0.05$, 2000 epochs) and plot their decision boundaries side by side in a 2D PCA projection. Report test accuracy for each.

> *Note: decision boundaries are visualized in a 2D PCA projection of the data, not in the original 30-dimensional feature space. This is standard practice — boundaries in high-dimensional space cannot be drawn directly.*

In your writeup: (i) compare the two boundaries and connect to model capacity; (ii) comment on the limitations of the PCA visualization given the variance explained (specifically: why might distinct classes appear overlapped in the 2D projection even if

they are separable in the 30D space?); (iii) given the medical context, would you prefer to minimize false negatives or maximize overall accuracy?

Part (g): Learning Rate Analysis — Breast Cancer (4 points)
Train the same architecture (`hidden_dim=` 16, 2000 epochs) with three learning rates: 0.001, 0.05, and 1.0. Plot all three loss curves on a single log-scale figure. In your writeup, describe the behaviour of each curve and connect it to the gradient descent discussion from Lecture 8.

Part (h): Depth vs Width — Breast Cancer (4 points)
Compare a wide shallow network (`layer_dims` $= [30, 64, 1]$, $\approx 2,049$ parameters) against a narrow deep network (`layer_dims` $= [30, 16, 16, 16, 1]$, $\approx 1,073$ parameters). Exact parameter equality is not required; the goal is comparable scale. Train both with `lr=` 0.05 for 2000 epochs using the provided `DeepMLP` class. Plot loss curves and decision boundaries. In your writeup, address: which converges faster; which achieves higher test accuracy; what the experiment suggests about depth vs width at matched parameter counts.

Note: Deep networks can be harder to optimize than shallow ones due to issues like vanishing gradients. If the deep network performs worse or converges more slowly, explicitly discuss why this might happen despite the comparable parameter count.

What to Submit

For theory problems:

1. Typeset all answers in a single PDF (handwritten solutions will not be accepted)
2. Show all derivation steps clearly; final answers without reasoning will receive little or no credit
3. Clearly label each question and part (e.g., `Q1(a)(ii)`)
4. Use LaTeX for mathematical expressions whenever possible

For coding problems:

1. Download `assignment2_coding.ipynb`
2. Complete all `TODO` sections in code cells
3. Add your written analysis and explanations in markdown cells
4. Run all cells top-to-bottom to ensure all outputs are displayed
5. Save and submit the completed `.ipynb` file

The notebook contains detailed instructions, starter code, hints, and expected outputs for each part.

BONUS QUESTIONS

*These questions are entirely optional. They are designed to reward deeper engagement with the material and may be used to earn additional points beyond the base score. You are **not required** to attempt them to achieve full credit on this assignment.*

Bonus Question 1: Naive Bayes as a Linear Classifier (10 points)

Throughout this course we have taken a *discriminative* approach to classification: we define a neural network, choose a loss function, and use gradient descent to find weights that make the posterior $p(C_k|x)$ as accurate as possible. The model never asks *why* the data looks the way it does—it just learns to draw boundaries.

But there is a completely different philosophy. A **generative** classifier asks: "What process produced this data?" It models the prior probability of each class $p(C_k)$ and the class-conditional density $p(\phi|C_k)$ describing what inputs look like given each class. The posterior then falls out automatically from Bayes' theorem—no gradient descent, no loss function, no iterations. Just counting.

What makes this question surprising is the punchline. Despite starting from completely different assumptions and using a completely different training procedure, the generative classifier produces a posterior that takes *exactly* the same softmax form as your network's output layer. Two roads, one destination. This question guides you through proving that from first principles.

Setup

Consider a K-class problem where the input $\phi \in R^M$ has M components, each taking L discrete states encoded as 1-of-L binary indicators $\phi_{il} \in \{0, 1\}$ with $\sum_{l=1}^{L} \phi_{il} = 1$. Under the **naive Bayes assumption**, features are independent given the class:

$$p(\phi|C_k) = \prod_{i=1}^{M} p(\phi_i|C_k) = \prod_{i=1}^{M} \prod_{l=1}^{L} \mu_{kil}^{\phi_{il}}$$

where $\mu_{kil} = p(\phi_{il} = 1|C_k)$ is the probability that the i-th feature takes state l given class C_k, with $\sum_{l=1}^{L} \mu_{kil} = 1$ for all k, i.

Let $\pi_k = p(C_k)$ denote the prior probability of class C_k, with $\sum_k \pi_k = 1$. We are given a training set $\{\phi_n, t_n\}$ for $n = 1, \ldots, N$ using 1-of-K coding: $t_{nk} = 1$ if example n belongs to class C_k. Let $N_k = \sum_n t_{nk}$.

Part (a): The Posterior is a Softmax (4 points)

The posterior under Bayes' theorem is:

$$p(C_k|\phi) = \frac{p(\phi|C_k)\,\pi_k}{\sum_j p(\phi|C_j)\,\pi_j} = \frac{exp(a_k)}{\sum_j exp(a_j)}, \qquad a_k = lnp(\phi|C_k) + ln\pi_k.$$

(i) Using the naive Bayes factorization, show that:

$$lnp(\phi|C_k) = \sum_{i=1}^{M} \sum_{l=1}^{L} \phi_{il} ln\mu_{kil}.$$

Hint: Take the log of the factorized density and use $ln \prod_l \mu_{kil}^{\phi_{il}} = \sum_l \phi_{il} ln\mu_{kil}$.

(ii) Substitute into a_k to show that it is **linear** in ϕ:

$$a_k = \sum_{i=1}^{M} \sum_{l=1}^{L} w_{kil}\phi_{il} + c_k, \qquad w_{kil} = ln\mu_{kil}, \quad c_k = ln\pi_k.$$

In 2–3 sentences, explain why this is surprising and what it reveals about the relationship between a generative classifier built by counting and the output layer of a neural network trained by gradient descent.

Part (b): Maximum Likelihood Estimates by Counting (4 points)

One of the most appealing properties of naive Bayes is that training requires no optimization at all. The maximum likelihood estimates for all parameters have closed-form solutions that amount to nothing more than counting examples.

(i) Show that maximizing the log-likelihood with respect to π_k subject to $\sum_k \pi_k = 1$ gives:

$$\hat{\pi}_k = \frac{N_k}{N}.$$

Hint: Introduce a Lagrange multiplier λ *for the constraint. Differentiate* $\sum_k N_k ln\pi_k + \lambda(\sum_k \pi_k - 1)$ *with respect to* π_k*, set to zero, and solve.*

(ii) Show that maximizing the log-likelihood with respect to μ_{kil} subject to $\sum_l \mu_{kil} = 1$ gives:

$$\hat{\mu}_{kil} = \frac{\sum_{n=1}^{N} t_{nk} \phi_{nil}}{N_k}.$$

Interpret this result in one sentence: what is $\hat{\mu}_{kil}$ literally counting, and why does the formula make immediate intuitive sense?

Part (c): Limitations and Connections (2 points)

No model is free from assumptions, and naive Bayes is no exception.

(i) The naive Bayes assumption—that features are independent given the class—is almost never exactly true in practice. Give one concrete example of a classification task where this assumption is clearly violated. Does violating it mean the classifier fails entirely, or can it still perform reasonably well? Explain briefly.

(ii) In Part (a) you showed that a_k is linear in ϕ. Does this mean naive Bayes can only solve linearly separable problems in the *original* input space? Explain carefully, keeping in mind that ϕ is a feature representation constructed from the raw input, not necessarily the raw input itself.

Bonus Question 2: Cost-Sensitive Classification (10 points)

Every classification system you have trained in this course has optimized for accuracy: the fraction of test examples it gets right. Accuracy is clean, easy to compute, and completely indifferent to *which* mistakes the model makes. In many real applications, that indifference is not a feature—it is a flaw.

Consider an AI system screening chest X-rays for lung cancer. Two types of mistakes are possible. A **false positive** flags a healthy patient as high-risk: they undergo an unnecessary follow-up, experience anxiety, and incur medical costs— stressful, but recoverable. A **false negative** clears a cancerous patient as healthy: they go home untreated while the disease progresses—potentially fatal. A model that achieves 95% accuracy by confidently clearing everyone is not a good model. It is a dangerous one.

Decision theory gives us the tools to encode this asymmetry directly into the classification rule. Rather than predicting the most probable class, we choose the action that minimizes the *expected cost* of being wrong—where the costs are specified explicitly by a **loss matrix** that reflects real-world consequences. This question guides you through deriving and applying that framework, and then asks you to connect it back to the network you trained in Part (f) of the coding section.

Setup

Consider a binary classification problem ($K = 2$) with classes C_1 = cancer and C_2 = healthy. The **loss matrix** L has entries $L_{kj} \geq 0$, where L_{kj} is the cost incurred when the true class is C_k and we predict C_j. Diagonal entries $L_{kk} = 0$ correspond to correct decisions. We use:

$$L = (0\ 100\ 1\ 0) \qquad (rows\ =\ true\ class,\ columns\ =\ predicted\ class)$$

The entry $L_{12} = 100$ encodes the catastrophic cost of a missed cancer diagnosis. The entry $L_{21} = 1$ encodes the much smaller cost of an unnecessary follow-up. Given posterior probabilities $p(C_k|x)$, the **expected loss** for predicting class C_j is:

$$\Delta_j(x) = \sum_{k=1}^{2} L_{kj}\, p(C_k|x).$$

The optimal rule predicts $j^* = argmin_j \Delta_j(x)$.

Part (a): The Optimal Decision Threshold (4 points)

(i) Write out $\Delta_1(x)$ and $\Delta_2(x)$ explicitly by substituting the numerical values from the loss matrix. Simplify using $L_{kk} = 0$.

(ii) Derive the condition under which the optimal decision is to predict cancer. Use $p(C_1|x) + p(C_2|x) = 1$ to express the rule purely in terms of $p(C_1|x)$ and show it takes the form:

$$Predict\ cancer\ if\ p(C_1|x) > \theta$$

Derive θ in terms of the general loss matrix entries L_{12} and L_{21}, then compute its numerical value for the given matrix. Compare to the standard threshold $\theta = 0.5$ and explain in one sentence why the direction of the shift makes intuitive sense given the costs.

(iii) What happens to θ as L_{12} increases relative to L_{21}? What does this imply about how aggressively the system should flag patients for follow-up?

Part (b): The Reject Option (4 points)

Even with a well-calibrated threshold, there will be cases where the model is genuinely uncertain—where $p(C_1|x)$ sits in a region where both errors are costly and the evidence is inconclusive. In high-stakes settings, forcing an automated decision in such cases may itself be the wrong choice. Instead, we can introduce a third action: **reject**, meaning defer the case to a human expert.

We model this by introducing a **rejection cost** $\lambda \geq 0$: deferring to a human incurs cost λ regardless of the true class, giving a constant expected risk $\Delta_{rej}(x) = \lambda$. The optimal decision now minimizes over three options: predict cancer, predict healthy, or reject.

(i) Suppose $\lambda = 10$. Compute the optimal decision for each case below. Show your calculations explicitly.

$p(C_1\|x)$	$p(C_2\|x)$	Optimal decision
0.99	0.01	
0.50	0.50	
0.08	0.92	
0.12	0.88	

(ii) Based on your completed table, describe the range of $p(C_1|x)$ that triggers rejection. Why is the rejection region *not* symmetric around $p(C_1|x) = 0.5$? Connect your answer to the asymmetry in the loss matrix.

Part (c): Connecting to Your Trained Network (2 points)

The network you trained on the Wisconsin Breast Cancer dataset in Part (f) of the coding section uses a fixed threshold of 0.5 to classify tumours as malignant or benign—equivalent to assuming the 0–1 loss matrix, where all mistakes are equally costly.

(i) Given the asymmetric costs developed in this problem, would you advocate for a different threshold when deploying that network in a real screening context? State what threshold you would use and justify it using the framework above.

(ii) The 0–1 loss matrix has $L_{kj} = 1$ for all $k \neq j$ and 0 on the diagonal. Show in one or two lines that minimizing expected loss under 0–1 loss reduces to predicting the most probable class. Then state concisely: under what real-world conditions is the 0–1 loss an appropriate choice, and when is it potentially dangerous?